

Learning from Click-Logs: A Two-Stage Re-Ranker for MIND and EB-NeRD

CS4.406 Information Retrieval & Extraction — Assignment 2 Design Note

1. What we built

Assignment 1 produced two candidate scorers: BM25 over titles and abstracts, and a dense retriever. Assignment 2 keeps those as **stage 1** and adds a **stage 2** LightGBM *lambdarank* model over engineered behavioural features, with the stage-1 score itself passed in as a feature so the ranker can learn how much to trust it per impression rather than having a fixed blend imposed on it.

The same code path scores stage 1 for training, validation and inference. That is a deliberate constraint rather than an implementation detail: the three bugs in Section 7 were all cases of the training and serving paths quietly disagreeing, and none of them raised an error.

Per-dataset instantiation

	MIND	EB-NeRD
Stage-1 encoder	all-MiniLM-L6-v2 (384d)	Ekstra Bladet Word2Vec (300d)
Training split	MIND-small train	ebnerd_small train
Test set	MINDlarge_test (2.37M)	ebnerd_testset (13.5M)
Features shipped	6 of 12	12 of 12
Language	English	Danish

Table 1. The pipeline is shared; only the encoder and the serving-safe feature set differ.

2. Click-history and session features (Q1)

Twelve features span three families. **Click history**: click count and an exponentially decayed recency sum (7-day half-life). **Session**: clicks in the last 1h and 24h, plus mean read time and scroll depth where the dataset records them. **Article**: popularity, freshness, a binary category match and a continuous category affinity (the share of the user's history in the candidate's category). Semantic similarity enters as the stage-1 cosine plus the maximum similarity to any single history article, which survives averaging over a user with several distinct interests.

Position bias: measured, then rejected

Q1.2 asks for position bias. Pooled over all impressions it looks real and strong: click-through rate falls monotonically with candidate position. It is an artifact. Impressions differ in candidate-list length, a longer list has a lower per-candidate click rate by construction, and only long lists reach deep positions, so deep positions are populated almost entirely by low-rate impressions.

Holding list length fixed removes the effect. The mean correlation between position and click rate within a fixed list length is EBNERD_DEMO -0.039, MIND +0.033 — indistinguishable from zero on both. Both datasets shuffle the in-view list, so no display-order signal exists to learn and a position feature would inject noise. We therefore report the measurement instead of the feature.

Behaviour-window boundary and a leak we found in our own code

Features for an impression at time T use only clicks strictly before T, enforced by binary search over per-user time-sorted arrays. Two subtler issues mattered more than the boundary itself. First, training originally used a user's *entire* history while MIND's test file exposes a truncated one; both are now capped at 50. Second, article popularity was counted over the whole history table, 54.5% of which falls inside the validation period. That is future information relative to the impressions being scored, and it inflated validation metrics. Popularity is now windowed to strictly before the validation period.

3. Re-ranker results, before and after (Q2)

MIND

System	AUC	MRR	nDCG@5	nDCG@10
Stage-1 only (before)	0.6514	0.3663	0.3500	0.4088
NRMS (reproduced baseline)	0.5842	0.3130	0.2908	0.3539
GBDT, behavioural only	0.5925	0.3105	0.2952	0.3590
GBDT + semantic (after)	0.6617	0.3820	0.3640	0.4247
GBDT, serving-safe	0.6621	0.3802	0.3630	0.4233

EB-NeRD

System	AUC	MRR	nDCG@5	nDCG@10
Stage-1 only (before)	0.4919	0.3082	0.3382	0.4272
NRMS (reproduced baseline)	0.5598	0.3509	0.3918	0.4686
GBDT, behavioural only	0.6818	0.4460	0.5046	0.5575
GBDT + semantic (after)	0.6812	0.4517	0.5097	0.5604
GBDT, serving-safe	0.6812	0.4517	0.5097	0.5604

Table 2. Per-impression metrics on held-out validation impressions. All systems are scored on the identical impression sample, which is what makes the paired tests in Section 4 valid.

The two datasets invert each other. On MIND the semantic signal carries the system: stage-1 alone already beats every behavioural-only variant, and the re-ranker's job is to add a modest amount on top of it. On EB-NeRD the Word2Vec document vectors are close to chance on their own, while behavioural features are strong enough to reach the final score almost unaided — freshness alone accounts for roughly 45% of the model's split gain, which suits a Danish tabloid where the article mix turns over fast.

That inversion is the single most useful observation we took from this assignment. The same architecture wins on both datasets for opposite reasons, so any conclusion about *which component matters* drawn from one dataset would not have transferred to the other. It is also why the ablation in Section 4 reaches different verdicts per dataset rather than one global one.

4. Baseline reproduced, then beaten (Q3)

The reproduced baseline is NRMS (Wu et al., 2019), the standard neural baseline for both leaderboards: multi-head self-attention over title word embeddings to encode an article, the same mechanism again over clicked articles to encode a user, and a dot product to score. Training follows the paper with negative sampling and softmax cross-entropy over one positive and K sampled negatives. We train from scratch rather than from GloVe and use a smaller embedding width, both to fit CPU-only training; the deviations are recorded in the checkpoint config. NRMS is scored on exactly the impressions the re-ranker is scored on.

MIND

Contrast	Metric	95% CI of difference	Excludes 0
improved - baseline	auc	[+0.0620, +0.0779]	yes
improved - baseline	ndcg5	[+0.0618, +0.0792]	yes
improved - baseline	ndcg10	[+0.0585, +0.0746]	yes
improved - stage1	auc	[+0.0059, +0.0142]	yes
improved - stage1	ndcg5	[+0.0085, +0.0188]	yes
improved - stage1	ndcg10	[+0.0112, +0.0200]	yes
improved - nrms	auc	[+0.0676, +0.0874]	yes
improved - nrms	ndcg5	[+0.0623, +0.0827]	yes
improved - nrms	ndcg10	[+0.0600, +0.0799]	yes
serving - improved	auc	[-0.0024, +0.0030]	no
serving - improved	ndcg5	[-0.0038, +0.0016]	no
serving - improved	ndcg10	[-0.0038, +0.0009]	no

EB-NeRD

Contrast	Metric	95% CI of difference	Excludes 0
improved - baseline	auc	[-0.0060, +0.0050]	no
improved - baseline	ndcg5	[-0.0028, +0.0117]	no
improved - baseline	ndcg10	[-0.0034, +0.0088]	no
improved - stage1	auc	[+0.1769, +0.1999]	yes
improved - stage1	ndcg5	[+0.1601, +0.1815]	yes
improved - stage1	ndcg10	[+0.1236, +0.1410]	yes
improved - nrms	auc	[+0.1090, +0.1334]	yes
improved - nrms	ndcg5	[+0.1052, +0.1293]	yes
improved - nrms	ndcg10	[+0.0815, +0.1011]	yes
serving - improved	auc	[+0.0000, +0.0000]	no
serving - improved	ndcg5	[+0.0000, +0.0000]	no
serving - improved	ndcg10	[+0.0000, +0.0000]	no

Table 3. Paired bootstrap over users, resampling both systems with the same draw. A gain is claimed only where the interval excludes zero.

The two-stage re-ranker beats the reproduced NRMS baseline on both datasets, and the interval excludes zero on every metric. We attach a caveat rather than claim more than the experiment supports: our NRMS is a reduced reproduction — embeddings learned from scratch instead of initialised from GloVe, a narrower hidden width and two epochs, all to fit CPU-only training. The published model reaches roughly 0.67 AUC on MIND, so part of this margin is a training budget rather than an architectural verdict.

An ablation that did not go our way

The claim we actually set out to test — that adding the stage-1 semantic score to the behavioural model improves it — holds on MIND and **fails on EB-NeRD**. On MIND the gain is large and the interval is far from zero. On EB-NeRD the interval for the same contrast straddles zero on all three metrics, so on that dataset the Word2Vec feature buys nothing measurable once behavioural features are present, consistent with its near-chance standalone AUC. Reporting it as a win on the strength of the MIND result would be exactly the kind of averaging the paired test exists to prevent.

The Q9 control behaves as intended. On MIND, dropping the six features the test set cannot reconstruct leaves accuracy statistically unchanged, which is the useful result: the shipped model gives up nothing by refusing to depend on signals it will not have. On EB-NeRD the difference is identically zero because that test bundle supplies every feature, so the serving-safe and full models are the same model.

5. Extended evaluation and slices (Q5)

Metrics are reported for the full two-stage pipeline over four slices: cold-start versus warm users (split at five clicks of history) and head versus tail articles (head = top 20% by training popularity). Intervals are bootstrap over users.

MIND — serving model

Slice	AUC	nDCG@5	nDCG@10	Novelty	Coverage	ILD
all	0.6621	0.3630	0.4233	15.19	0.0205	0.7848
cold	0.6045	0.3487	0.4088	15.25	0.0097	0.8440
warm	0.6744	0.3661	0.4264	15.18	0.0192	0.7720
head	0.6504	0.5615	0.6080	11.80	0.0054	0.8826
tail	0.6650	0.4061	0.4689	16.90	0.0185	0.7832

EB-NeRD — serving model

Slice	AUC	nDCG@5	nDCG@10	Novelty	Coverage	ILD
all	0.6812	0.5097	0.5604	16.92	0.0684	0.7733
cold	0.5883	0.3919	0.4807	16.43	0.0085	0.7325
warm	0.6817	0.5103	0.5608	16.93	0.0679	0.7734
head	0.6457	0.7659	0.7815	8.87	0.0126	0.7192
tail	0.6791	0.5197	0.5695	17.26	0.0596	0.7762

Table 4. Accuracy and beyond-accuracy metrics by slice.

Cold-start users are the weakest slice on both datasets, which is the expected shape: with little history the user vector is poorly determined and the behavioural features degenerate toward constants. The head/tail split shows the opposite concern — head articles are easier to rank but contribute the least novelty, so a system tuned purely on accuracy drifts toward recommending what is already popular.

6. Serving and scale (Q4)

	Index	Cands	p50	p95	p99	QPS/core	\$/1k
MIND	217 MB	35.3	10.7	12.8	14.0	71.5	\$0.00013
EBNERD_SMALL	258 MB	11.7	12.3	14.1	15.2	65.9	\$0.00014

Table 5. Single-request latency measured end to end against the real trained model, not a stand-in. Cost assumes a 2-vCPU on-demand node and capacity planned at p99.

Both datasets meet a p99 < 100 ms budget on a single node, with feature extraction rather than model scoring as the dominant cost.

7. Three bugs that did not raise an error

Each of these produced plausible output and a finished submission, which is precisely why they survived. We record them because the failure mode — a silent disagreement between two code paths — is the same in all three.

Stage-1 score inert during training. The training frame filled the stage-1 column with a constant 0.0 placeholder. A zero-variance feature cannot be split on, so LightGBM listed it in feature_names while having learned nothing from it, and the real values it met at inference changed no prediction. Training

now asserts the column has non-zero variance and aborts if not.

The highest-gain feature was dead at serving. `article_popularity` carried 78% of the MIND model's split gain but was hardcoded to an empty map at inference, so every candidate scored 0. With freshness also unusable (MIND has no publication dates) and the remaining features constant within an impression, the shipped model was effectively ranking on a single binary category flag. Popularity is now derived from the test file's own history and rank-normalised so the two corpora are comparable.

Every confidence interval was wrong. Bootstrap resampling duplicates a drawn user's rows. Those copies kept their original `impression_id`, so `groupby(impression_id)` merged them into one oversized impression instead of counting them separately. MRR and nDCG were biased low enough that point estimates fell outside their own intervals. AUC is computed flat across rows and is invariant to proportional duplication, so it looked correct throughout — which is what hid the bug. Each draw now gets its own impression ids, and four regression tests assert the estimate lies inside its interval.

8. Where this breaks at 10x

Latency is not the constraint. Candidate scoring is $O(\text{candidates})$ per request rather than $O(\text{catalogue})$, because both leaderboards hand us the candidate list; a ten-fold larger catalogue grows the embedding matrix but leaves per-request latency flat.

User history is the first thing to break. It is held as an in-process pandas frame. At 10x users this is gigabytes per node, and because every node needs its own copy, memory grows with throughput as well as with data. It has to move to a key-value store keyed by user, which converts a memory problem into a network round-trip inside the latency budget.

Feature extraction, not the model, is the hot path. It is the majority of end-to-end time on both datasets, while GBDT scoring is a minority. Effort spent on a larger model would buy less than effort spent on precomputing per-user aggregates, which are identical across a user's impressions.

Retrieval is the real 10x question. Reported `recall@200` for stage-1 is low on both datasets. That is tolerable only because the candidate list is given. A deployment that must retrieve from the full catalogue would need an approximate index and would be bounded by stage-1 recall long before stage-2 precision mattered.

Training dilutes before serving does. The MIND model is fitted on 20k sampled impressions and the EB-NeRD model on a small bundle, then applied to test sets of 2.37M and 13.5M impressions over far more users than training ever saw. Scaling the training sample is a larger lever than any architectural change at this point.

9. Leaderboard submissions

	MIND	EB-NeRD
Test impressions	2,370,727	13,536,710
Model shipped	6-feature serving model	12-feature serving model
Throughput	2,040 imp/s	2,040 imp/s
Wall time	24 min	111 min
Archive	<code>mind_large_reranker_submission.zip</code>	<code>ebnerd_large_reranker_submission.zip</code>

Table 6. Both archives were produced by the streaming inference scripts and validated line-by-line before upload.

Neither test set fits in memory once candidates are expanded, so both scripts stream: MIND line-by-line from `behaviors.tsv`, EB-NeRD one parquet row group at a time, with scores flushed in batches and user history held as flat offset-indexed arrays rather than per-user objects. The EB-NeRD history index is cached, which cuts a nine-minute startup to one second on re-runs.

Both scripts refuse to emit a file whose line count differs from the number of impressions, and every input line produces exactly one output line even when an impression has no candidates. This matters because the graders align predictions to impressions positionally: a file that is short by one line is not rejected, it is silently scored against the wrong impressions, which would look like a modelling failure rather than a formatting one.

10. Reproducing this

```
python build_pipeline.py → run_bm25.py → run_embeddings.py → run_reranker.py →  
run_nrms.py → run_evaluation.py → run_serving_analysis.py →  
src/scripts/run*_inference.py
```

Each model is written beside a `.features.json` recording the exact feature order it was trained with, and inference refuses to run if that list disagrees with the booster. Tests under `tests/` assert the behaviour-window boundary, train/serve feature parity, and bootstrap correctness.